

DOMAĆI ZADATAK: PROBLEM TRGOVAČKOG PUTNIKA (TSP)

Fakultet inženjerskih nauka, Univerzitet u Kragujevcu

Studijski program: Računarska tehnika i softversko inženjerstvo

Predmet: Veštačka inteligencija (IV godina, VIII semestar)

Školska godina: 2025/2026.

Student: Anita Mijatović

Broj indeksa: 612/2022

Mesto i datum: Kragujevac, maj 2026. godine

1. Opis zadatka i definicija problema

Zadatak zahteva rešavanje Problema Trgovačkog Putnika (TSP — *Travelling Salesman Problem*) na grafu od 5 gradova: A, B, C, D i E. Između gradova postoje definisane putanje čije su dužine zadate u matrici rastojanja.

Cilj je, polazeći iz grada A, pronaći minimalan Hamiltonov ciklus koji:

- Posećuje svaki od preostalih gradova (B, C, D, E) tačno jednom,
- Vraća putnika nazad u polazni grad A,
- Minimizuje ukupnu dužinu pređenog puta.

Za rešavanje problema primenjuju se dve različite heurističke funkcije kroz pohlepnu pretragu, a dobijeni rezultati se na kraju porede sa egzaktnim rešenjem.

1.1 Matrica rastojanja

Ulazni graf ima 5 čvorova i definisan je sledećom matricom rastojanja (gde vrednost inf označava da ne postoji direktna veza između gradova):

	A	B	C	D	E
A	inf	7	6	10	13
B	7	inf	7	inf	10
C	6	7	inf	8	9
D	10	inf	8	inf	6
E	13	10	9	6	inf

Napomena iz grafa: Graf nije kompletan — iz matrice se jasno vidi da ne postoji direktna veza između čvorova B i D (inf). Ova specifičnost direktno utiče na donošenje odluka u algoritmima pretrage.

2. Teorijska osnova

2.1 Formalizacija TSP-a i Prostor Stanja

TSP spada u klasu NP-teških kombinatornih optimizacionih problema. Za n gradova, broj mogućih jedinstvenih Hamiltonovih ciklusa iznosi $(n-1)! / 2$. Za graf od 5 gradova, taj broj je $(5-1)! / 2 = 12$ jedinstvenih ruta. Egzaktno rešenje se može pronaći iscrpnom pretragom (*Brute Force*), ali pošto složenost raste faktorijelno, za veće probleme (gde je $n > 20$) to postaje računski neizvodljivo, pa se uvode heurističke metode.

Prostor stanja za ovaj problem formalno se definiše kao:

- **Stanje:** (trenutni grad, skup posećenih gradova)
- **Početno stanje:** $(A, \{A\})$
- **Cilj:** $(A, \{A, B, C, D, E\})$ — svi gradovi su posećeni i uspešno je izvršen povratak u početni grad A.
- **Operator:** Pomeranje iz trenutnog u neposećeni grad sa kojim postoji direktna veza (težinska ivica).
- **Cena puta:** Suma rastojanja svih pređenih ivica u ciklusu.

2.2 Pohlepna pretraga (Greedy Best-First Search)

Pohlepna pretraga je algoritam koji u svakom koraku bira lokalno optimalan potez na osnovu vrednosti heurističke funkcije $h(n)$, potpuno ignorišući dosadašnju cenu pređenog puta $g(n)$.

Formula glasi: $f(n) = h(n)$.

Karakteristike pohlepne pretrage:

- **Kompletnost:** Nije garantovana (algoritam se može zaglaviti u lokalnom optimumu ili naići na slepu ulicu).
- **Optimalnost:** Nije garantovana.
- **Vremenska i prostorna složenost:** U najgorem slučaju iznosi $O(b^m)$, gde je b faktor grananja, a m maksimalna dubina prostora stanja. Međutim, dobra heuristika drastično smanjuje broj stanja koja se ispituju u praksi.

3. Trag izvršavanja heurističkih funkcija

3.1 H1: Nearest Neighbor Heuristika (Lokalni pristup)

Definicija i algoritam

Heuristička funkcija H1 procenjuje vrednost čvora n isključivo na osnovu rastojanja od trenutnog grada do najbližeg susednog, još uvek neposećenog grada: $h1(n) = \min \{ \text{dist}(n, v) \mid v \text{ ne pripada posećenim gradovima} \}$.

Algoritam se izvršava kroz sledeće korake:

1. Postavi $\text{start} = A$, $\text{posećeni} = \{A\}$, $\text{ruta} = [A]$, $\text{cena} = 0$.
2. Sve dok ima neposećenih gradova, izračunaj $h1(v)$ za sve susedne neposećene gradove i izaberi onaj sa minimalnom distancom.
3. Dodaj izabrani grad u rutu i obeleži ga kao posećen.
4. Kada se obiđu svi gradovi, zatvori ciklus povratkom u grad A.

Trag izvršavanja u kodu (H1)

Prateći tačnu logiku iz koda, algoritam generiše sledeći trag:

Korak	Trenutni grad	Kandidati (grad: dist)	Izabran (h min)	Deo rute
Korak 1	A	B: 7, C: 6 , D: 10, E: 13	C (h=6)	A → C
Korak 2	C	B: 7 , D: 8, E: 9	B (h=7)	A → C → B
Korak 3	B	D: inf, E: 10	E (h=10)	A → C → B → E
Korak 4	E	D: 6	D (h=6)	A → C → B → E → D
Zatvaranje	D	Povratak u A (dist=10)	A	A → C → B → E → D → A

- **Pronađena H1 ruta:** A → C → B → E → D → A
- **Ukupna cena:** 6 + 7 + 10 + 6 + 10 = 39

3.2 H2: Greedy Edge Insertion Heuristika (Globalni pristup)

Definicija i algoritam

Za razliku od lokalnog pristupa, heuristika H2 donosi odluke globalno. Ona posmatra sve ivice u grafu istovremeno. Vrednost svake ivice $e=(u,v)$ je njena dužina: $h2(e) = \text{dist}(u,v)$.

Algoritam sortira sve ivice celog grafa u rastući redosled i pohlepno ih ubacuje u skup izabranih ivica, strogo poštujući dva strukturna uslova za formiranje ispravnog Hamiltonovog ciklusa:

- **Uslov 1 (Stepen):** Nijedan čvor ne sme dostići stepen veći od 2, jer se u svaki grad ulazi i iz njega izlazi tačno jednom.
- **Uslov 2 (Ciklus):** Ivica ne sme da zatvori prevremeni krug (mali podciklus). Prevremeni krug se detektuje pomoću *Union-Find* strukture podataka. Krug je dozvoljen isključivo u poslednjem koraku kada se spajaju svi gradovi.

Trag izvršavanja u kodu (H2)

Ivice se sortiraju po dužini, a odluke o prihvatanju se donose redom:

Ivica	Rastojanje	Status u kodu / Obrazloženje
A-C	6	• ODABRANA (Stepeni OK, nema ciklusa)
D-E	6	• ODABRANA (Stepeni OK, nema ciklusa)
A-B	7	• ODABRANA (Stepeni OK, nema ciklusa)
B-C	7	• ODBAČENA (Formira prevremeni krug A-C-B-A pre

Ivica	Rastojanje	Status u kodu / Obrazloženje
		nego što su svi gradovi uključeni)
C-D	8	• ODABRANA (Stepeni OK, spaja komponente)
C-E	9	• ODBAČENA (Čvor C već ima stepen 2 od ivica A-C i C-D)
A-D	10	• ODBAČENA (Čvor A već ima stepen 2 od ivica A-C i A-B)
B-E	10	• ODABRANA (Zatvara finalni ciklus od svih 5 gradova)

- **Odabrane ivice:** A–C (6), D–E (6), A–B (7), C–D (8), B–E (10)
- **Rekonstruisana H2 ruta (od čvora A):** $A \rightarrow C \rightarrow D \rightarrow E \rightarrow B \rightarrow A$
- **Ukupna cena:** $6 + 8 + 6 + 10 + 7 = 37$

4. Referentno optimalno rešenje (Brute Force)

Radi validacije i procene kvaliteta predloženih heuristika, izvršena je iscrpna pretraga (*Brute Force*) svih mogućih permutacija gradova. Za 5 gradova, prostor pretrage obuhvata 12 jedinstvenih ciklusa (odnosno 24 permutacije ako se računa i smer kretanja).

Neke od ključnih testiranih ruta i njihove ciklne cene:

- $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A \Rightarrow 7 + 10 + 8 + 6 + 13 = 44$
- $A \rightarrow C \rightarrow B \rightarrow E \rightarrow D \rightarrow A \Rightarrow 6 + 10 + 10 + 6 + 10 = 42$
- $A \rightarrow C \rightarrow E \rightarrow D \rightarrow B \rightarrow A \Rightarrow 6 + 9 + 6 + \text{inf} = \text{inf}$ (Nemoguća ruta, nema veze D-B)
- $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C \rightarrow A \Rightarrow 7 + 10 + 6 + 8 + 6 = 37$ (★ **OPTIMUM**)

Globalni teoretski optimum iznosi **37**. Njemu je ekvivalentan i obrnuti smer kretanja ($A \rightarrow C \rightarrow D \rightarrow E \rightarrow B \rightarrow A$), koji takođe daje cenu 37.

5. Poređenje i analiza rezultata

Na osnovu izvršenih proračuna i simulacija iz koda, kreirana je zbirna tabela performansi:

Algoritam	Ruta	Cena	Odstupanje od opt.	Status
Brute Force (Optimum)	$A \rightarrow B \rightarrow E \rightarrow D \rightarrow C \rightarrow A$	37	0.0%	Teoretski minimum
H1: Nearest Neighbor	$A \rightarrow C \rightarrow B \rightarrow E \rightarrow D \rightarrow A$	39	+5.4%	Suboptimalno
H2: Greedy	$A \rightarrow C \rightarrow D \rightarrow E$	37	0.0%	★ OPTIMALNO

Algoritam	Ruta	Cena	Odstupanje od opt.	Status
Edge	→ B → A			

5.1 Analiza ponašanja H1 (Nearest Neighbor)

Algoritam H1 je pokazao slabost već u drugom koraku. Kada se našao u čvoru C, lokalno najbliži neposećeni grad bio je B ($d=7$), dok je grad D bio za nijansu dalji ($d=8$). Odabirom grada B, algoritam je napravio pohlepnu lokalnu odluku koja ga je kasnije primorala da plati visoku cenu povratka iz D u A ($d=10$).

Ovo je klasičan primer "**problema poslednje milje**" kod Nearest Neighbor heuristike — donošenje kratkovidih, jeftinih odluka na početku ostavlja nepovoljne i skupe alternative za sam kraj rute. Na većim grafovima, ova heuristika u proseku daje rezultate koji su za 20-25% lošiji od optimalnih. Prednost joj je niska složenost $O(n^2)$ i brza implementacija.

5.2 Analiza ponašanja H2 (Greedy Edge Insertion)

Algoritam H2 je uspešno izbegao lokalne zamke jer je problem posmatrao sa globalnog nivoa. Prve tri najjeftinije ivice u celom grafu su bile A-C (6), D-E (6) i A-B (7) i sve su uspešno prihvaćene. Ključni momenat se desio kod četvrte ivice B-C (7) — iako je bila jeftina, algoritam ju je odbacio jer bi zatvorila izolovani krug A-C-B-A. Zbog toga je uzeo sledeću najbolju ivicu C-D (8).

Pregledom cele slike grafa pre povlačenja poteza, H2 je prirodno rekonstruisao optimalni Hamiltonov ciklus. Korišćenjem *Union-Find* strukture, provera ciklusa se obavlja izuzetno brzo u $O(\alpha(n))$ vremenu, što je čini računski vrlo efikasnom.

6. Zaključak

Na testiranoj instanci sa 5 gradova, **heuristika H2 (Greedy Edge Insertion) se pokazala kao superiornija**, jer je pronašla tačan globalni optimum (cena 37), dok je heuristika H1 završila u suboptimalnom stanju sa odstupanjem od +5.4% (cena 39).

Generalni teorijski zaključci o izboru heuristika za TSP:

1. **H2 (Globalni pristup)** je stabilnija i robusnija heuristika. Pošto bira ivice na osnovu sortirane liste celog grafa, daleko je manje podložna lokalnim ekstremima i anomalijama u rasporedu grana. Na složenijim i većim instancama grafova, statistički pruža znatno manji procenat odstupanja od egzaktnog rešenja.
2. **H1 (Lokalni pristup)** je opravdan jedino kod izrazito malih grafova ($n \leq 10$) ili u sistemima gde su resursi memorije i procesora toliko kritični da se zahteva najjednostavnija moguća logika pretrage bez potrebe za sortiranjem i praćenjem struktura grafovskih komponenti.

Za potrebe rada sa velikim industrijskim grafovima, logičan korak napred bio bi nadogradnja H2 heuristike algoritmima lokalnog poboljšanja (poput *2-opt* ili *3-opt* optimizacija) ili prelazak na metaheuristike kao što su genetski algoritmi i simulirano kaljenje.